

APPENDIX: DADS DEMONSTRATION WALKTHROUGH

In this section, we walk through our demonstration (video), covering the complete steps for installing and using our tool in a clean environment. To run through our demo, the user can download the virtual machine as the DADSVm.rar file found [here](#). Then, the user can launch the virtual machine in Virtualbox, open a terminal, and go to the /home/xqfu/thrift directory. Next, the user can start from Step 4 below—the first three steps are described below for the completeness purpose, as users who would like to start from scratch in their fresh environment would need those steps to set up before they can run what we show in the demo. For more details, please refer to the README document mentioned in Step 2 below.

Note that in this demo we show that we ran the example subject to clearly illustrate the three-phase design of our tool.

1. Check prerequisites (Step 1)

First, please make sure the following environments are configured properly:

- Ubuntu workstation (We used Ubuntu 16.04 LTS)
- JDK 1.8 (We used java-1.8.0-openjdk-adm64)

2. Download the DADS tool (Step 2)

To obtain and install our tool, please follow the steps below:

- Download the open-source [tool package](#).
- Confirm that there is a "tool" directory that contains several jars (these are all the libraries required), and a "data/Thrift-java" directory that contains the (calculator) application developed using Thrift.
- Check a "code" directory, which includes the source code of DADS and convenience scripts for running our tool against Apache Thrift as used in our evaluation study.
- Finally, view a detailed README document, which includes all the information here in this appendix, and more.

3. Download and install an example subject (Step 3)

We demonstrate the use of our tool against a subject Apache Thrift, for which the following steps can be followed:

- In a terminal, create a directory for the subject, say "thrift", and a subdirectory "0110".
- Download Thrift file thrift-0.11.0.tar.gz from [here](#) to the "thrift/0110" directory.
- Unzip Thrift file: tar -zxvf thrift-0.11.0.tar.gz
- Copy all shell script files from the sub-directory "code/shell/Thrift" of the tool package obtained in Step 1 above to the "thrift" directory.
- Check inside each of the scripts to make sure that the "libs" related directories actually contain all the required libraries (jars) as mentioned in Step 2 above with respect to the user's actual environment. In particular, set the ROOT variable to the right directory (in this case, it should be the parent directory of the "thrift" directory).
- Copy the entire "data/Thrift/java" directory from the tool package to the "thrift" directory.

The following steps show how to run DADS against the installed Apache Thrift subject.

4. Run DADS (Step 4)

4.1 DADS phase 1: Instrumentation. In phase 1 of DADS, we execute "/DADSInstr.sh" to instrument Thrift for monitoring first/last events of all executed methods. After the execution, the instrumented program files are in the sub-folder "DADSInstrumented", shown in Figure 4.

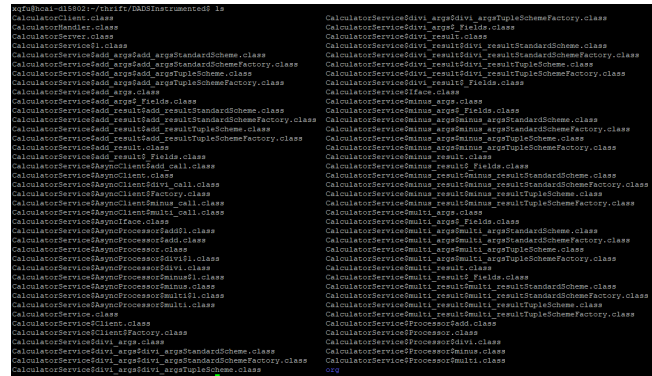


Figure 4: The results of DADS phase 1: Instrumented program files

4.2. DADS phase 2: Arbitration and Adjustment. In phase 2 of DADS, we first set milliseconds (e.g., 4,000) for a user-specified budget constraint in the file *budget.txt*, shown in Figure 5.



Figure 5: The operations of DADS phase 2: The setting for the user-specified budget constraint

Then, we execute "/serverDADS.sh" and "/clientDADS.sh" separately, to launch the server and client component of Thrift, respectively, as shown in Figure 6.

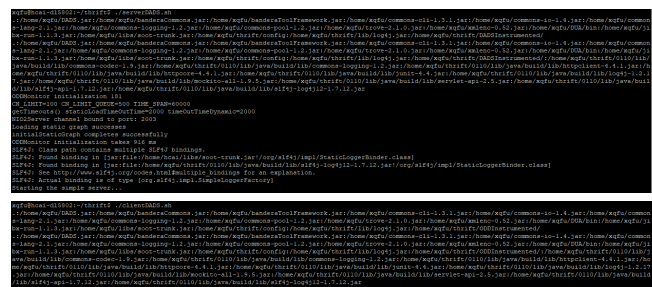


Figure 6: The operations of DADS phase 2: starting the server (top) and client (bottom) component of the example subject system Thrift.

Next, the client automatically sends computation information (operands and basic arithmetic operations such as addition, subtraction, multiplication, and division) to the server and gets the computation results from the server, as shown in Figure 7.

```

client.multi(7,9)=63
client.divi(7,9)=0
The client will sleep 3 seconds
client.add(17,81)=98
client.minus(17,81)=-64
client.multi(17,81)=1377
client.divi(17,81)=0
The client will sleep 7 seconds
client.add(62,73)=135
client.minus(62,73)=-11
client.multi(62,73)=4526
client.divi(62,73)=0
The client will sleep 3 seconds

```

Figure 7: The operations of DADS phase 2: The operations in the client

```

Event computation took 16296 ms
Q table is updated in x:3,y:15,action=2, new value=0.45
oldconfigurations: 111111 newConfigurations: 101111
oldStaticConfigurations: 11 newStaticConfigurations: 10

```

Figure 8: The results of DADS phase 2: The changing analysis configurations

```

xqfu@hcai-d15802:~/thrift$ ./DADSQueryClient.sh
The network address localhost:2000 cannot be connected!
The network address localhost:2001 can be connected.
The network address localhost:2002 cannot be connected!
The network address localhost:2003 cannot be connected!
The network address localhost:2004 can be connected.
The network address localhost:2005 cannot be connected!
The network address localhost:2006 cannot be connected!
The network address localhost:2007 cannot be connected!
The network address localhost:2008 cannot be connected!
The network address localhost:2009 cannot be connected!
Please input method:
<org.apache.thrift.transport.TSocket: void open()>

```

Figure 9: The operations of DADS phase 3: The operations in the query client

```

<CalculatorServiceMinus_args: org.apache.thrift.scheme.IScheme scheme(org.apache.thrift.protocol.TProtocol)>
<org.apache.thrift.protocol.TBinaryProtocol: void writeMessageBegin(org.apache.thrift.protocol.TMessage)>
<CalculatorClient: void main(java.lang.String[])>
<CalculatorServiceMinus_result: org.apache.thrift.scheme.IScheme getScheme()>
<CalculatorServiceAdd_args: CalculatorServiceAdd_args setMulti(int)>
<org.apache.thrift.TMultiplexer: int setMulti(int,int,boolean)>
<CalculatorServiceMinus_args: void write(org.apache.thrift.protocol.TProtocol)>
<org.apache.thrift.TMultiplexer: int setMulti(int,int,boolean)>
<CalculatorServiceAdd_result: java.lang.String getFieldName()>
<CalculatorServiceAdd_result: Fields: CalculatorServiceAdd_result.Fields() values()>
<CalculatorServiceAdd_result: void write(org.apache.thrift.protocol.TProtocol)>
<CalculatorServiceMinus_result: void write(org.apache.thrift.protocol.TProtocol)>
<org.apache.thrift.transport.TTransport: int readAll(Byte[] buf,int,int)>
<org.apache.thrift.transport.TSocket: void open()>
<CalculatorServiceAdd_result: void read(org.apache.thrift.protocol.TProtocol,org.apache.thrift.TBase)>
<CalculatorServiceMinus_result: int add(int,int)>
<CalculatorServiceMinus_args: void setMultiSet(boolean)>
Query took 2016 ms

```

Figure 10: The results of DADS phase 3: The returned slice (method-level dependencies) in the query client

We see that analysis configurations are changing during the execution according to a Q-learning strategy, shown in Figure 8.

4.3. DADS phase 3: User Interaction. In phase 3 of DADS, we start a query client to connect the Thrift server and client, and send a method name as a slicing query, shown in Figure 9.

Then, we get the result, the method-level dependencies as the slice, shown in Figure 10. These dependencies are used as impact set as in an impact analysis supporting other tasks like regression testing. Furthermore, these dynamic dependencies can be used for any other applications based on such dependencies. We also note that the user received the dependence results in 2016 ms (as shown at the last line of Figure 10), which is within 4 seconds (4,000 ms), the budget constraint set earlier.